

Para Estudo

Baseado no material da Prof. Kenia Parente Lopes Mendonça

1. Revisão de Algoritmos

1.1 Saída de Dados (`print`)

O comando `print()` é usado para exibir mensagens ou o conteúdo de variáveis no console (terminal).

```
# Exemplo 1: Imprimindo texto (string)
```

```
print("Olá, mundo!")
```

```
# Exemplo 2: Imprimindo variáveis
```

```
nome = "Maria"
```

```
idade = 25
```

```
print(nome, idade)
```

```
# Exemplo 3: F-strings (Forma moderna de formatar)
```

```
# Coloque um 'f' antes das aspas e as variáveis entre {}
```

```
print(f"O nome da aluna é {nome} e sua idade é {idade}.")
```

1.2 Variáveis e Tipos de Dados

Variáveis são "contêineres" para armazenar valores. Python é uma linguagem de tipagem dinâmica, o que significa que você não precisa declarar o tipo; ele é inferido automaticamente.

- **`str` (String):** Texto. Ex: `"Olá"`, `'Python'`
- **`int` (Inteiro):** Números inteiros. Ex: `10`, `-5`, `2025`
- **`float` (Ponto Flutuante):** Números com casas decimais. Ex: `1.5`, `-10.99`

- **bool (Booleano):** Valor lógico. Ex: `True`, `False` (Sempre com 'T' e 'F' maiúsculos)

```
nome_aluno = "João"      # str
media_final = 9.5         # float
idade = 19                # int
aprovado = True           # bool
```

1.3 Operadores

Aritméticos

- `+` (Adição)
- `-` (Subtração)
- `*` (Multiplicação)
- `/` (Divisão - sempre retorna `float`)
- `//` (Divisão Inteira - descarta a parte decimal)
- `%` (Módulo - resto da divisão)
- `**` (Exponenciação - elevado à potência)

```
print(10 / 3)    # Saída: 3.333...
print(10 // 3)   # Saída: 3
print(10 % 3)    # Saída: 1 (resto)
print(2 ** 3)    # Saída: 8 (2*2*2)
```

Comparação (Retornam `True` ou `False`)

- `==` (Igual a)
- `!=` (Diferente de)
- `>` (Maior que)
- `<` (Menor que)
- `>=` (Maior ou igual a)
- `<=` (Menor ou igual a)

Lógicos (Combinam expressões booleanas)

- `and` (E lógico - Ambas as condições devem ser `True`)
- `or` (OU lógico - Pelo menos uma condição deve ser `True`)

- `not` (NÃO lógico - Inverte o valor)

```
idade = 20
possui_cnh = False

# Exemplo de 'and'
pode_dirigir = (idade >= 18) and (possui_cnh == True)
print(pode_dirigir) # Saída: False
```

1.4 Entrada de Dados (`input`)

O comando `input()` pausa o programa e espera o usuário digitar algo. **Crucial:**

`input()` SEMPRE retorna o valor como uma **string** (`str`).

Você deve converter o tipo manualmente usando `int()` , `float()` , etc.

```
# 1. Pede o nome (str)
nome = input("Digite seu nome: ")

# 2. Pede a idade (str) e converte para int
idade_str = input("Digite sua idade: ")
idade_int = int(idade_str)

# 3. Forma direta (mais comum)
nota = float(input("Digite sua nota: "))

# 4. Cálculo
proxima_idade = idade_int + 1
print(f"Olá, {nome}. Ano que vem você terá {proxima_idade} anos.")
```

2. Estruturas de Controle

2.1 Condicionais (`if` / `elif` / `else`)

Executa blocos de código diferentes com base em uma condição ser `True` ou `False`. A indentação (espaços no início da linha) é obrigatória e define o bloco.

```
media = float(input("Digite a média final: "))

if media >= 7.0:
    print("Aprovado!")
    print("Parabéns!")
elif media >= 4.0:
    print("Recuperação.")
else:
    print("Reprovado.")

print("Fim do programa.") # Este 'print' está fora do bloco, sempre
```

2.2 Laço (`while`)

O `while` (enquanto) repete um bloco de código **enquanto** uma condição for verdadeira. É ideal para quando não sabemos quantas vezes vamos repetir.

Exemplo 1: Contador Simples

```
cont = 0
while cont < 5:
    print(f"O número é {cont}")
    cont = cont + 1 # IMPORTANTE: Atualizar a variável de controle

print("Laço 'while' encerrado.")
```

Exemplo 2: Condição de Parada (Flag)

```
numero = -1 # Inicializa com um valor que entra no laço

while numero != 0:
    numero = int(input("Digite um número (ou 0 para sair): "))
    print(f"Você digitou {numero}")
```

```
print("Você digitou 0. Fim.")
```

2.3 Laço (**for**)

O **for** (para) é usado para iterar sobre uma sequência (como uma lista, string ou um intervalo de números).

Exemplo 1: Iterando sobre uma Lista

```
frutas = ["Maçã", "Banana", "Uva"]

for fruta in frutas:
    print(f"Eu gosto de {fruta}")
```

Exemplo 2: Usando **range()**

A função **range()** gera uma sequência de números. É a forma mais comum de repetir algo N vezes.

```
# range(5) gera 0, 1, 2, 3, 4
for i in range(5):
    print(f"Repetição número {i}")

# range(1, 6) gera 1, 2, 3, 4, 5 (começa em 1, para ANTES do 6)
for i in range(1, 6):
    print(f"Contando: {i}")
```

3. Estruturas de Dados

3.1 Listas (Vetores Unidimensionais)

Listas são coleções ordenadas e mutáveis de itens. Elas são o equivalente a "vetores" ou "arrays" em outras linguagens. São definidas por colchetes `[]`.

```
# 1. Criando uma lista
notas = [9.5, 7.0, 8.5, 10.0]
nomes = ["Ana", "Bruno", "Carla"]
lista_vazia = []

# 2. Acessando itens (índice começa em 0)
print(nomes[0])    # Saída: Ana
print(notas[2])    # Saída: 8.5

# 3. Alterando um item
nomes[1] = "Bruno Costa"

# 4. Adicionando um item (no final)
nomes.append("Daniel")

# 5. Removendo um item
nomes.pop()        # Remove o último (Daniel)
nomes.remove("Ana") # Remove pelo valor "Ana"

# 6. Tamanho da lista
tamanho = len(nomes)
print(f"A lista tem {tamanho} nomes.")

# 7. Iterando com 'for' (muito comum)
for nota in notas:
    print(nota)
```

3.2 Listas (Matrizes Bidimensionais)

Matrizes em Python são simplesmente "listas dentro de listas".

```
# Exemplo do PDF (Cidades do Tocantins)
cidades = [
    ["Paraiso", 1963], # Linha 0
    ["Gurupi", 1958], # Linha 1
    ["Palmas", 1989], # Linha 2
    ["Miranorte", 1964] # Linha 3
]
```

```
# Acessando dados ( [linha][coluna] )
print(cidades[2][0]) # Saída: Palmas (Linha 2, Coluna 0)
print(cidades[0][1]) # Saída: 1963 (Linha 0, Coluna 1)

# Iterando sobre a matriz com 'for'
for cidade_info in cidades:
    # cidade_info é uma lista, ex: ["Paraiso", 1963]
    nome = cidade_info[0]
    ano = cidade_info[1]
    print(f"A cidade de {nome} foi fundada em {ano}.")

# Desempacotamento direto (mais "Pythônico")
for nome, ano in cidades:
    print(f"{nome} ({ano})")
```

3.3 Dicionários (dict)

Dicionários são coleções **não ordenadas** (em versões antigas do Python) de pares **chave-valor**. Eles são definidos por chaves `{ }`. Ideais para armazenar dados de um objeto (ex: aluno, produto).

```
# 1. Criando um dicionário
aluno = {
    "nome": "Carla",
    "matricula": "202403",
    "nota_final": 9.8,
    "ativo": True
}

# 2. Acessando valores (pela CHAVE)
print(aluno["nome"]) # Saída: Carla
print(aluno["nota_final"]) # Saída: 9.8
# Forma mais segura (não dá erro se a chave não existir)
print(aluno.get("curso", "Não informado")) # Saída: Não informado

# 3. Adicionando ou Alterando valores
aluno["nota_final"] = 10.0 # Altera valor existente
aluno["curso"] = "Sistemas de Informação" # Adiciona novo par

# 4. Iterando sobre um dicionário
```

```
for chave, valor in aluno.items():  
    print(f"{chave}: {valor}")
```

```
# Saída:  
# nome: Carla  
# matricula: 202403  
# ...etc
```

4. Funções (def)

Funções são blocos de código reutilizáveis que executam uma tarefa. Elas são definidas com a palavra-chave `def`.

- **Parâmetros:** Variáveis que a função *recebe*.
- **return:** O valor que a função *devolve*.

```
# 1. Definindo uma função simples
```

```
def saudacao(nome):  
    print(f"Olá, {nome}! Bem-vindo(a).")
```

```
# 2. Definindo uma função que retorna um valor
```

```
def somar(a, b):  
    resultado = a + b  
    return resultado
```

```
# --- Programa Principal ---
```

```
# 3. Chamando (executando) as funções
```

```
saudacao("Ana") # Chama a função, que vai imprimir "Olá, Ana!..."
```

```
valor_soma = somar(10, 5) # Chama a função e armazena o retorno
```

```
print(f"O resultado da soma é {valor_soma}") # Saída: 15
```

<  >

5. Manipulação de Arquivos (I/O)

Python pode ler e escrever em arquivos de texto (`.txt` , `.csv` , etc.) de forma simples.

A forma mais segura de fazer isso é com o gerenciador de contexto `with open()` , que garante que o arquivo será fechado automaticamente, mesmo se ocorrer um erro.

Escrevendo em um Arquivo (`'w'` e `'a'`)

- **Modo `'w'` (write - escrita):** Cria um novo arquivo (ou apaga um existente) e escreve conteúdo nele.
- **Modo `'a'` (append - anexar):** Adiciona conteúdo ao final de um arquivo existente, sem apagar o que já estava lá.

```
dados = "Esta é a primeira linha.\nEsta é a segunda linha.\n"
```

```
# 1. Escrevendo (Modo 'w' - apaga o anterior)
```

```
with open("meu_arquivo.txt", "w", encoding="utf-8") as arquivo:  
    arquivo.write(dados)
```

```
# 2. Anexando (Modo 'a' - adiciona ao final)
```

```
with open("meu_arquivo.txt", "a", encoding="utf-8") as arquivo:  
    arquivo.write("Esta linha foi anexada.\n")
```

Lendo de um Arquivo (`'r'`)

O **Modo `'r'` (read - leitura)** é usado para ler o conteúdo de um arquivo.

- `.read()` : Lê o arquivo inteiro como uma única string.
- `.readline()` : Lê apenas a próxima linha.
- `.readlines()` : Lê todas as linhas e as retorna como uma lista de strings.

```
# 1. Tenta abrir o arquivo para leitura
```

```
try:
```

```
    with open("meu_arquivo.txt", "r", encoding="utf-8") as arquivo:  
        # Exemplo A: Ler tudo de uma vez  
        conteudo_completo = arquivo.read()  
        print("--- Conteúdo Completo ---")
```

```
print(conteudo_completo)

# Para ler linhas (precisa 'voltar' ao início do arquivo)
arquivo.seek(0)

# Exemplo B: Ler linha por linha (ideal para arquivos grandes)
print("\n--- Lendo Linha por Linha ---")
for linha in arquivo:
    print(linha.strip()) # .strip() remove o '\n' do final da linha

# 2. Captura o erro se o arquivo não existir
except FileNotFoundError:
    print("ERRO: O arquivo 'meu_arquivo.txt' não foi encontrado.")
except Exception as e:
    print(f"Ocorreu um erro inesperado: {e}")
```

6. Exercícios Resolvidos

Exercício 1: Média com **while** (Condição de Parada/Flag)

Crie um programa que solicite ao usuário a digitação de números inteiros. O programa deve parar quando o usuário digitar 0 (zero). No final, o programa deve exibir a média dos números digitados (excluindo o zero).

```
# Inicializa as variáveis de controle
soma_total = 0
contador = 0

print("Digite números para calcular a média (Digite 0 para sair)")

while True:
    try:
        numero = int(input(f"Digite o {contador + 1}º número: "))
    except:
        print("Valor inválido. Digite um número inteiro.")
        continue # Pula para a próxima iteração
```

```
# Verifica a condição de parada (flag)
if numero == 0:
    break # Encerra o laço 'while'

soma_total += numero
contador += 1

if contador > 0:
    media = soma_total / contador
    print(f"\nA média dos {contador} números foi: {media}")
else:
    print("\nNenhum número foi digitado (além do 0).")
```

Exercício 2: Iterando em Listas com **for** (Maior Valor)

Considere uma lista de compras, onde cada item é uma sub-lista contendo o nome do cliente e o valor da compra. Crie um programa que encontre e imprima o nome do cliente que fez a maior compra e o valor dela.

```
compras = [
    ["Ana", 150.75],
    ["Bruno", 520.00],
    ["Carla", 95.50],
    ["Daniel", 890.99],
    ["Elisa", 320.00]
]

if not compras: # Verifica se a lista não está vazia
    print("Lista de compras vazia.")
else:
    maior_valor = compras[0][1]
    cliente_maior_compra = compras[0][0]

    for nome_cliente, valor_compra in compras:
        if valor_compra > maior_valor:
            maior_valor = valor_compra
            cliente_maior_compra = nome_cliente
```

```
print(f"Valor da maior compra: R$ {maior_valor}; cliente:
```

Exercício 3: Funções e Matrizes (Cidades do Tocantins)

(Baseado no Exercício 3 da pág. 91 do PDF) Crie um programa com um menu interativo que utilize três funções: `imprimir_cidades()`, `verificar_data(ano)` e `acrescentar_cidade()`, usando a lista de cidades global.

```
idades = [
    ["Paraiso", 1963], ["Gurupi", 1958], ["Palmas", 1989], ["P
]

def imprimir_cidades():
    print("\n--- Lista de Cidades ---")
    for nome, ano in cidades:
        print(f"Cidade: {nome}; Ano de criação: {ano}")

def verificar_data(ano_usuario):
    print(f"\n--- Verificando ano: {ano_usuario} ---")
    for nome, ano_criacao in cidades:
        if ano_criacao < ano_usuario:
            print(f"A cidade de {nome} foi criada ANTES de {ar
        elif ano_criacao > ano_usuario:
            print(f"A cidade de {nome} foi criada DEPOIS de {a
        else:
            print(f"A cidade de {nome} foi criada EXATAMENTE e

def acrescentar_cidade():
    print("\n--- Adicionar Nova Cidade ---")
    nome = input("Digite o nome da nova cidade: ")
    try:
        ano = int(input(f"Digite o ano de fundação de {nome}:
        cidades.append([nome, ano])
        print(f"Cidade {nome} ({ano}) adicionada com sucesso!")
    except:
        print("Ano inválido. Operação cancelada.")
```

```
# --- Programa Principal (Menu) ---
while True:
    print("\n--- MENU CIDADES (1: Listar, 2: Verificar Ano, 3:
    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        imprimir_cidades()
    elif opcao == "2":
        try:
            ano_teste = int(input("Digite o ano para comparar:
            verificar_data(ano_teste)
        except:
            print("Ano inválido.")
    elif opcao == "3":
        acrescentar_cidade()
    elif opcao == "0":
        print("Saindo...")
        break
    else:
        print("Opção inválida.")
```

Exercício 4: Dicionário Alinhado (Lista de Dicionários)

Crie um sistema de cadastro de alunos. O sistema deve armazenar os alunos em uma **lista de dicionários**. Cada aluno (dicionário) deve ter: "nome", "matricula" e "notas" (uma lista de floats). Crie funções para:

1. Adicionar um novo aluno.
2. Listar todos os alunos.
3. Calcular e exibir a média de um aluno específico (buscando pela matrícula).

```
# Estrutura de dados principal: Lista de Dicionários
cadastro_alunos = [
    {
        "nome": "Ana Silva",
        "matricula": "1001",
        "notas": [8.5, 9.0, 10.0]
    },
    {
```

```
        "nome": "Bruno Costa",
        "matricula": "1002",
        "notas": [6.0, 7.5, 5.0]
    }
]

def adicionar_aluno():
    print("\n--- Adicionar Aluno ---")
    nome = input("Nome do aluno: ")
    matricula = input("Matrícula: ")

    # Validação de notas
    notas = []
    try:
        n1 = float(input("Nota 1: "))
        n2 = float(input("Nota 2: "))
        n3 = float(input("Nota 3: "))
        notas = [n1, n2, n3]
    except:
        print("Erro nas notas. Operação cancelada.")
        return

    # Cria o novo dicionário do aluno
    novo_aluno = {
        "nome": nome,
        "matricula": matricula,
        "notas": notas
    }

    # Adiciona o dicionário à lista principal
    cadastro_alunos.append(novo_aluno)
    print(f"Aluno {nome} adicionado com sucesso!")

def listar_alunos():
    print("\n--- Lista de Alunos ---")
    for aluno in cadastro_alunos:
        # Itera sobre a lista de dicionários
        print(f"Nome: {aluno['nome']} (Mat: {aluno['matricula']})")
        print(f"  Notas: {aluno['notas']}")

def calcular_media_aluno():
    print("\n--- Calcular Média ---")
```

```
matricula_busca = input("Digite a matrícula do aluno: ")

aluno_encontrado = None
# Procura o aluno na lista
for aluno in cadastro_alunos:
    if aluno["matricula"] == matricula_busca:
        aluno_encontrado = aluno
        break # Para o 'for' assim que achar

if aluno_encontrado:
    nome = aluno_encontrado["nome"]
    notas = aluno_encontrado["notas"] # Pega a lista de notas
    media = sum(notas) / len(notas)
    print(f"A média do aluno {nome} é: {media:.2f}")
else:
    print(f"Erro: Aluno com matrícula {matricula_busca} não encontrado")

# --- Programa Principal (Menu) ---
while True:
    print("\n--- CADASTRO (1: Listar, 2: Adicionar, 3: Média, 4: Sair)")
    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        listar_alunos()
    elif opcao == "2":
        adicionar_aluno()
    elif opcao == "3":
        calcular_media_aluno()
    elif opcao == "4":
        print("Saindo...")
        break
    else:
        print("Opção inválida.")
```

Exercício 5: Manipulação de Arquivo (Salvar e Ler)

Crie um programa com duas funções:

1. *salvar_cidades_arquivo(lista_cidades)* : Recebe uma lista (matriz) de

idades e a salva em um arquivo `idades.txt`. Cada sub-lista deve virar uma linha no arquivo, com os dados separados por vírgula (formato CSV simples).

2. `ler_cidades_arquivo()`: Lê o arquivo `idades.txt`, o reconstrói na memória e o imprime no console.

```
# Dados iniciais
idades_para_salvar = [
    ["Paraiso", 1963],
    ["Gurupi", 1958],
    ["Palmas", 1989]
]

NOME_ARQUIVO = "idades_tocantins.txt"

def salvar_cidades_arquivo(lista_cidades):
    print(f"\nTentando salvar {len(lista_cidades)} cidades em
    try:
        with open(NOME_ARQUIVO, "w", encoding="utf-8") as f:
            for nome, ano in lista_cidades:
                # Formata a linha como: "Nome,Ano\n"
                linha = f"{nome},{ano}\n"
                f.write(linha)
            print("Arquivo salvo com sucesso!")
    except Exception as e:
        print(f"Erro ao salvar o arquivo: {e}")

def ler_cidades_arquivo():
    print(f"\nLendo dados do arquivo '{NOME_ARQUIVO}'...")
    lista_lida = []
    try:
        with open(NOME_ARQUIVO, "r", encoding="utf-8") as f:
            for linha in f:
                # 1. Remove espaços/quebras de linha extras
                linha_limpa = linha.strip() # Ex: "Palmas,1989"

                # 2. Divide a linha pela vírgula
                partes = linha_limpa.split(",") # Ex: ["Palmas", "1989"]

                # 3. Reconstrói os dados (convertendo o ano para inteiro)
                if len(partes) == 2: # Garante que a linha não é vazia
                    nome, ano_str = partes
                    ano = int(ano_str)
                    lista_lida.append([nome, ano])
```



```
        nome = partes[0]
        ano = int(partes[1])
        lista_lida.append([nome, ano])

# Imprime os dados lidos
print("Dados lidos do arquivo:")
for nome, ano in lista_lida:
    print(f"    - {nome} ({ano})")

except FileNotFoundError:
    print(f"ERRO: O arquivo '{NOME_ARQUIVO}' não foi encor
except Exception as e:
    print(f"Erro ao ler o arquivo: {e}")

# --- Programa Principal ---
# 1. Salva os dados no arquivo
salvar_cidades_arquivo(cidades_para_salvar)

# 2. Lê os dados de volta do arquivo
ler_cidades_arquivo()
```